

UrbanEye

AI-Smart Complaint and Civic Issue Resolution System

(ES-452: Major Project - Dissertation)

Submitted in partial fulfillment of the requirement for the award of the
degree of

Bachelor of Technology

in

Information Technology / Computer Science

Prepared by

Alok Ranjan

Enrollment No.: _____

Under the supervision of

Assistant Professor

Department of Information Technology / Computer Science

Institute Name: _____

May 2026

DECLARATION

I hereby declare that the dissertation titled “**UrbanEye: AI-Smart Complaint and Civic Issue Resolution System**” is the result of my own work carried out for the Major Project course. This work has been prepared in accordance with the academic guidelines prescribed for the degree program. I further declare that this report has not been submitted, either in full or in part, to any other university or institution for the award of any degree or diploma. Wherever the ideas, findings, or words of other researchers have been used, due acknowledgement has been made in the appropriate sections of the report.

Alok Ranjan

Student

Date: _____

CERTIFICATE

This is to certify that the dissertation work titled “**UrbanEye: AI-Smart Complaint and Civic Issue Resolution System**” has been carried out by **Alok Ranjan** under my supervision and guidance in partial fulfillment of the requirements for the award of the Bachelor of Technology degree. To the best of my knowledge, the work presented in this report is original and has not been submitted elsewhere for any other degree or diploma.

Project Supervisor

Head of Department

ACKNOWLEDGEMENT

The completion of this project has been possible only because of the support, encouragement, and guidance I received from many people. I would like to express my sincere gratitude to my project supervisor for continuous guidance, valuable suggestions, and constructive criticism at every stage of the work. Their insight helped convert a simple application idea into a structured system that addresses a real civic problem.

I am equally thankful to the faculty members of the department for providing the academic environment and technical foundation required to carry out this project. Their classes and discussions contributed significantly to my understanding of software engineering, full-stack development, and intelligent system design.

I would also like to acknowledge my friends and peers who helped with testing, feedback, and discussion during the design and implementation phases. Their practical observations improved the usability of the web and mobile interfaces and helped identify gaps in the complaint workflow.

Finally, I would like to thank my family for their constant encouragement and patience throughout the project. Their support gave me the motivation to complete the work with sincerity and confidence.

ABSTRACT

UrbanEye is an AI-assisted civic issue reporting and complaint resolution platform designed to improve the way citizens interact with local civic administration. Traditional complaint workflows are usually fragmented, slow, and poorly tracked. Citizens often register issues through separate channels, receive little visibility into progress, and rarely get structured public communication once a complaint is escalated. UrbanEye addresses this challenge through a unified digital system that supports complaint reporting, categorization, routing, status monitoring, and AI-generated public update content.

The system combines a web application for dashboard access, a mobile application for field-friendly reporting and tracking, a backend API for business logic and persistence, and a FastAPI-based AI service for complaint analysis. When a complaint is filed, the platform captures the title, description, location, and optional image. The AI layer performs rule-based triage with support for language-model-based enhancement, assigns category and priority, suggests the responsible department, and prepares an X-ready social update for official communication workflows.

The platform supports multiple user-facing views, including complaint dashboards, map hotspots, complaint details, administrative statistics, and profile management. The mobile application additionally provides an improved status-centric interface in which residents can quickly determine whether their complaints are pending, in progress, or resolved. A lightweight persistence strategy is used to support both online and fallback operation, allowing the platform to remain demonstrable even when external services are unavailable.

UrbanEye has been developed using Express.js, Prisma, PostgreSQL-ready schemas, FastAPI, React Native with Expo, and Next.js. The project demonstrates how full-stack application engineering and practical AI augmentation can be combined to solve a real-world civic communication and service-tracking problem in a simple, scalable, and user-friendly manner.

LIST OF FIGURES

Figure No.	Figure Title
3.4.1	Use Case Diagram of UrbanEye
3.4.2	Level 0 Data Flow Diagram
3.4.3	Level 1 Complaint Processing DFD
4.1.1	Work Breakdown Structure
4.2.1	System Architecture Diagram
4.2.2	Module Interaction Diagram
4.3.1	Activity Diagram for Complaint Submission
4.3.2	Activity Diagram for AI Analysis and Status Tracking
4.4.1	ER Diagram
4.4.2	Class / Data Model Diagram
5.1.1	Web Home and Dashboard Flow
5.1.2	Mobile Home and Dashboard Flow

LIST OF TABLES

Table No.	Table Title
1.1	Major Objectives of UrbanEye
2.1	Problems in Conventional Civic Complaint Workflows
3.1	Functional Requirements
3.2	Non-Functional Requirements
3.3	Tools, Technologies, and Platforms Used
4.1	Core Modules of the Proposed System
6.1	Testing Summary

TABLE OF CONTENTS

Declaration	I
Certificate	II
Acknowledgement	III
Abstract	IV
List of Figures and List of Tables	V
Chapter 1: Introduction	1
Chapter 2: Problem Statement	5
Chapter 3: Analysis	9
Chapter 4: Design and Architecture	17
Chapter 5: Implementation	31
Chapter 6: Testing	42
Chapter 7: Summary and Conclusion	47
Chapter 8: Limitations and Future Work	50
Bibliography	53
Appendix	54

CHAPTER 1: INTRODUCTION

1.1 Project Overview

UrbanEye is a full-stack civic complaint management platform that brings together complaint reporting, routing, analytics, AI-assisted triage, and user-facing status tracking in a single system. The core idea behind the project is simple: residents should be able to report a civic problem in one place, receive meaningful visibility into what happens next, and trust that the complaint is being directed to the correct department. At the same time, authorities need a lightweight operational view of complaint trends, priorities, and hotspots so that action can be better coordinated.

The project was developed as a multi-surface application instead of a single website because civic complaint handling naturally involves different stakeholders and environments. The web application supports dashboard-driven review and broader oversight, the mobile application supports quick complaint entry and resident-side tracking, the Express backend provides a structured API and persistence layer, and the FastAPI AI service produces triage intelligence and social communication text. This split makes the solution modular while keeping the user workflow coherent.

1.2 Need for the Project

In ordinary civic workflows, complaint handling suffers from four recurring issues: fragmented reporting channels, weak classification, poor transparency, and delayed communication. A resident may know that an issue such as garbage overflow, streetlight failure, drainage blockage, or water leakage exists, but may not know which department is responsible or whether the complaint has moved beyond initial registration. In many cases, the complaint is acknowledged but not clearly tracked from the citizen's perspective.

UrbanEye was designed to reduce that uncertainty. By classifying complaints into categories such as sanitation, electricity, water, and roads, the system supports better routing. By maintaining complaint state fields such as pending, in progress, and resolved, the platform gives the resident a more interpretable progress trail. By producing structured dashboards and hotspot views, it also helps turn individual complaints into actionable operational visibility.

1.3 Major Objectives

Objective	Description
Unified Complaint Lifecycle	Provide reporting, analysis, tracking, and public update preparation in one connected system.

AI-Assisted Triage	Automatically infer category, priority, department, suggested action, and social communication text.
Multi-Platform Access	Support both web and mobile access so that residents and operators can use the system conveniently.
Administrative Visibility	Present complaint counts, status breakdowns, and map hotspots for monitoring and follow-up.
Status Transparency	Allow residents to clearly see whether their complaint is pending, being handled, or solved.
Extensibility	Keep the architecture open for future integration with real social publishing, maps, and department workflows.

1.4 Scope of the Project

The scope of UrbanEye includes user registration and login, complaint submission with text and optional image, AI-assisted complaint analysis, dashboard and hotspot monitoring, mobile complaint tracking, and basic administrative analytics. The system is intended as a robust academic prototype with production-ready patterns in several modules, especially in API structure, data modeling, and fallback handling.

The present scope does not yet include field workforce dispatch automation, actual X publishing credentials, GIS mapping integration beyond a simplified hotspot representation, or a complete role-based workflow engine for department authorities. However, the current architecture is designed in a way that such features can be introduced incrementally.

Files in project 87,935	Code lines 358,025 total	Platforms Web, Mobile, Backend, AI
-----------------------------------	------------------------------------	--

CHAPTER 2: PROBLEM STATEMENT

2.1 Problem Definition

Civic service complaint systems often fail not because people are unwilling to report issues, but because the path from complaint to resolution is unclear. A user may file a complaint and then lose sight of what happens next. The responsible department may not be obvious, complaints may not be categorized consistently, and administrators may not have an immediate cross-channel view of the workload. This creates dissatisfaction for residents and inefficiency for authorities.

The problem becomes more serious when manual handling is involved. Without structured category labels, a water complaint may be mixed with road maintenance tasks. Without a priority signal, urgent incidents may be delayed. Without a public communication layer, residents may assume inaction even when internal processing has started. Thus, the core problem is both operational and communicative.

2.2 Existing System Drawbacks

Issue	Description	Impact
Fragmented Reporting	Residents use different channels or ad hoc mechanisms to submit civic issues.	Records become inconsistent and hard to track.
Weak Classification	Complaints are often stored as plain text without structured category or priority labels.	Wrong routing and delayed action.
Poor Status Visibility	Users are not clearly informed whether the issue is open, under action, or resolved.	Low trust and repeated follow-up requests.
No Communication Support	Public-facing updates require manual drafting every time.	Slow information flow and inconsistent messaging.
Limited Administrative Insight	Authorities cannot easily view hotspots, departmental breakdowns, or recent patterns.	Reactive instead of proactive decision-making.

2.3 Proposed Solution

UrbanEye proposes a layered digital system in which complaints move through a structured lifecycle. The user submits a complaint through web or mobile. The backend forwards the essential textual content to an AI service that performs local classification and can optionally enhance output using an LLM. The enriched complaint, along with its department, priority, suggested action, and X-ready

social update, is then stored or returned through the backend. The same complaint becomes visible in user dashboards, hotspot summaries, and administrative views.

Instead of treating AI as a separate showcase feature, the project uses it to reduce practical friction in the complaint-routing workflow. The AI component is therefore directly tied to utility: it shortens triage time, standardizes categorization, and supports communication readiness.

2.4 Specific Objectives

1. To build a complaint registration interface for citizens on both web and mobile platforms.
2. To design a backend API capable of storing, retrieving, and presenting complaint status information.
3. To implement an AI-assisted analysis module for category, priority, department, and social update generation.
4. To provide an administrative dashboard with counts, breakdowns, and complaint hotspot visibility.
5. To improve trust and transparency through resident-facing complaint status tracking.
6. To keep the system modular enough for future integration with maps, notifications, and social publishing.

CHAPTER 3: ANALYSIS

3.1 Software Requirement Specification

3.1.1 Functional Requirements

ID	Requirement	Description
FR-1	User Authentication	The system shall allow residents to register and log in through the backend authentication API.
FR-2	Complaint Submission	The system shall accept title, description, location, and optional image for complaint creation.
FR-3	Complaint Analysis	The AI service shall produce category, priority, department, sentiment, confidence, suggested action, and social post.
FR-4	Status Tracking	The user shall be able to view complaint status as pending, in progress, or resolved.
FR-5	Admin Analytics	The system shall provide aggregate counts and complaint distribution for administrative review.
FR-6	Hotspot Map Data	The system shall expose complaint hotspot information for locality-based visibility.
FR-7	X Account Preference	The mobile application shall allow the user to save a preferred X handle for update readiness.
FR-8	Offline/Fallback Support	The applications shall return usable fallback data when external services are unavailable.

3.1.2 Non-Functional Requirements

Category	Requirement
Usability	Interfaces should be simple enough for residents while still exposing meaningful admin insight.
Performance	Complaint analysis and dashboard loading should remain responsive for ordinary payload sizes.

Maintainability	Frontend, backend, and AI services should remain separable and independently maintainable.
Scalability	The data model and API patterns should support migration from mock storage to PostgreSQL-backed persistence.
Reliability	The system should continue functioning in development or degraded service mode using local fallback logic.
Portability	The product should be accessible through both browser and mobile interfaces.

3.2 Feasibility Study

Technical Feasibility

The project is technically feasible because its components rely on stable and well-documented technologies. Express.js is suitable for API development, Next.js is effective for dashboard-driven web interfaces, Expo supports rapid React Native development, and FastAPI is well suited for lightweight AI microservices. The use of Prisma with a PostgreSQL-ready schema further improves maintainability and future expansion.

Operational Feasibility

The proposed system is operationally feasible because the target workflow is understandable to both citizens and authorities. Residents only need to submit a description and location, while authorities or administrators benefit from pre-computed analytics. The AI component reduces manual interpretation rather than introducing extra complexity.

Economic Feasibility

The development stack used in UrbanEye is cost-effective for an academic and prototype-to-deployment transition. Most components are open source or have student-friendly plans. The architecture can initially run in a demo mode using local/mock data and later be upgraded to managed storage and hosted deployment when necessary.

3.3 Tools / Technologies / Platform Used

Layer	Tools and Technologies	Purpose
Web Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS	Citizen/admin dashboard, complaint views, reporting UI

Mobile Frontend	React Native, Expo, AsyncStorage	Resident complaint reporting and tracking on mobile devices
Backend API	Node.js, Express.js, Prisma, bcrypt	Authentication, complaint APIs, admin APIs, map APIs
AI Service	FastAPI, Pydantic, optional LangChain-based LLM integration	Complaint triage and social post generation
Persistence	Prisma schema, PostgreSQL-ready models, mock store, AsyncStorage	Complaint/user storage and local fallback persistence
Auxiliary Modules	Expo Image Picker, fetch, REST APIs	Image intake, network communication, mobile submission flow

3.4 Use Case Diagrams / Data Flow Diagrams

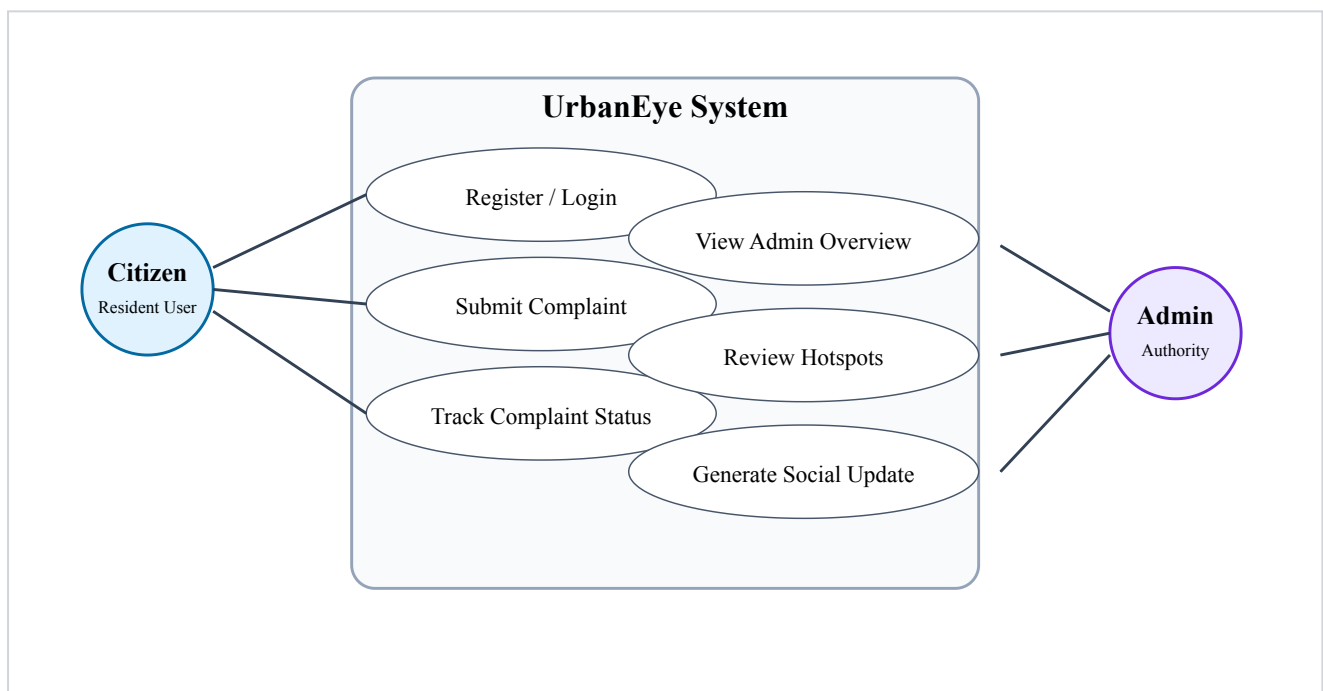


Figure 3.4.1: Use Case Diagram of UrbanEye

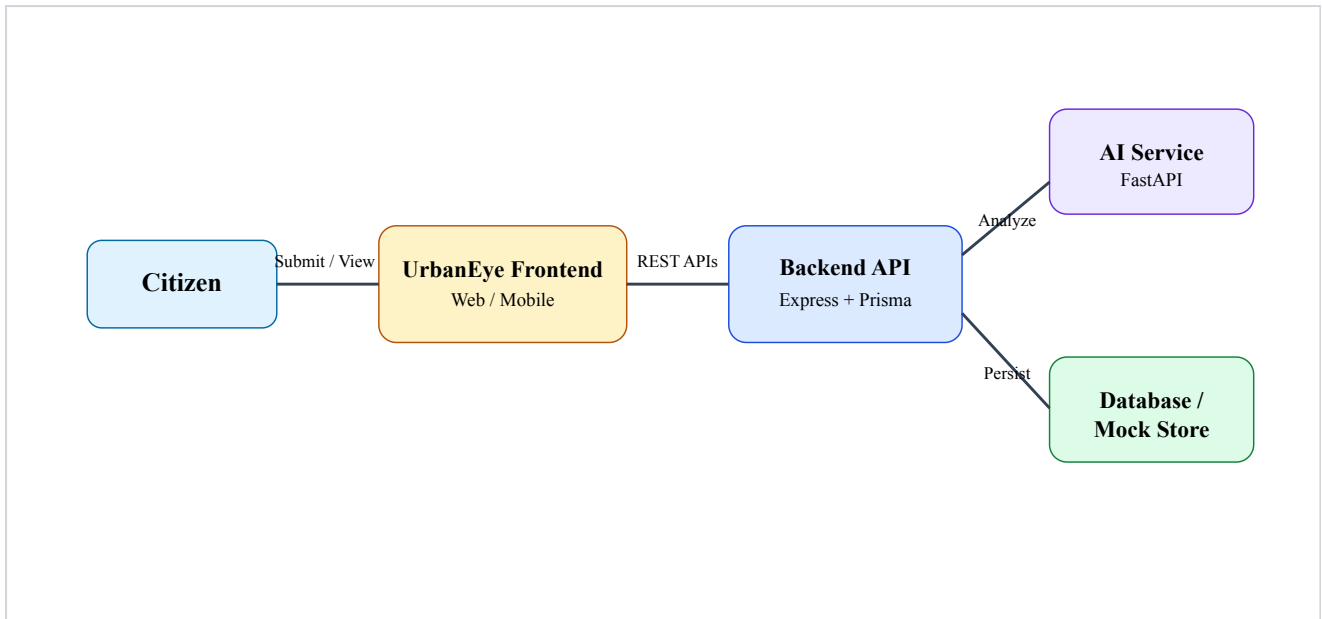


Figure 3.4.2: Level 0 Data Flow Diagram

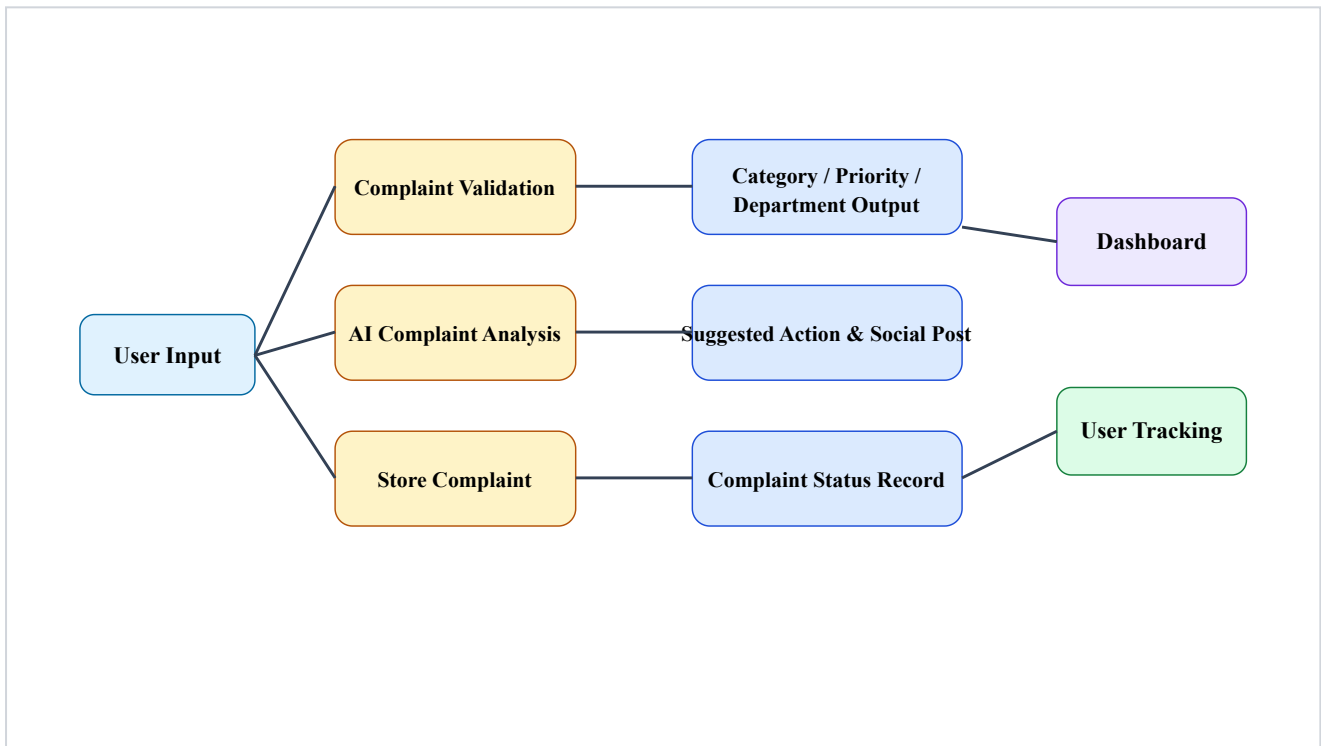


Figure 3.4.3: Level 1 Complaint Processing DFD

CHAPTER 4: DESIGN AND ARCHITECTURE

4.1 Structure Chart / Work Breakdown Structure

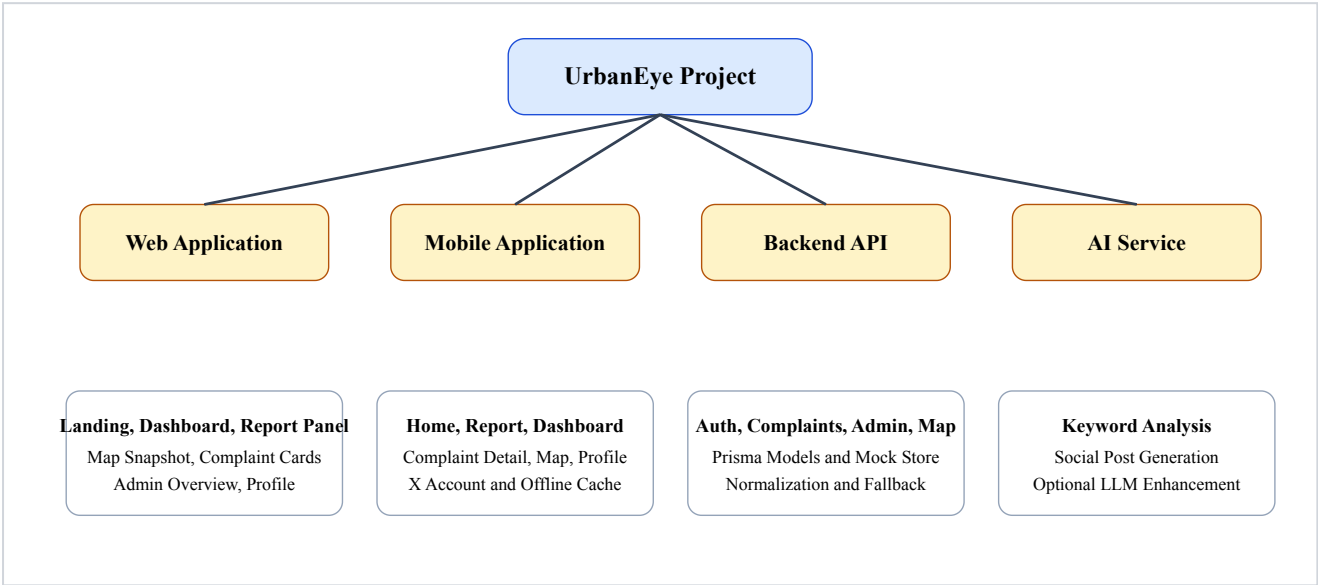


Figure 4.1.1: Work Breakdown Structure

4.2 Explanation of Modules

Module	Description
Authentication Module	Handles resident registration and login through Express routes and bcrypt-protected user records.
Complaint Management Module	Creates complaints, fetches complaint lists and details, and normalizes complaint shape for all clients.
AI Analysis Module	Determines category, priority, department, sentiment, suggested action, and X-ready social post text.
Administrative Dashboard Module	Provides summary counts, priority breakdowns, department distributions, and recent complaint snapshots.
Map Hotspot Module	Exposes complaint latitude/longitude and locality data for hotspot visualization.
Mobile Tracking Module	Shows complaint resolution progress, local persistence, and X handle preferences for residents.

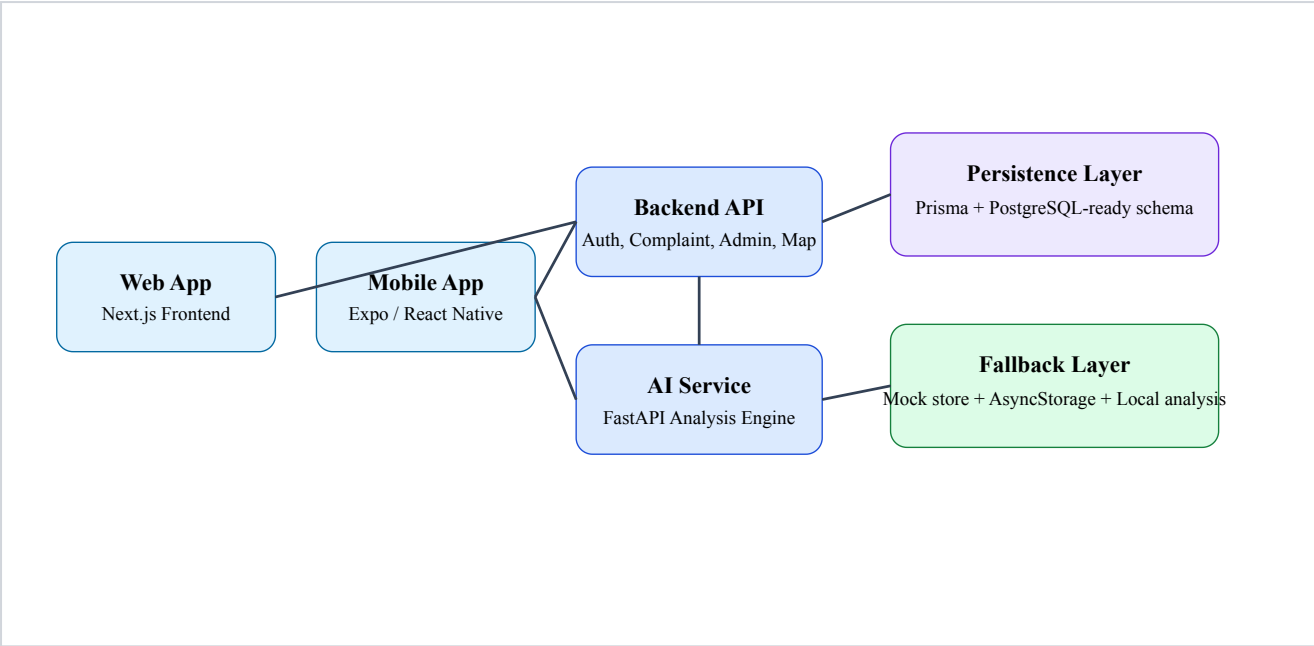


Figure 4.2.1: System Architecture Diagram

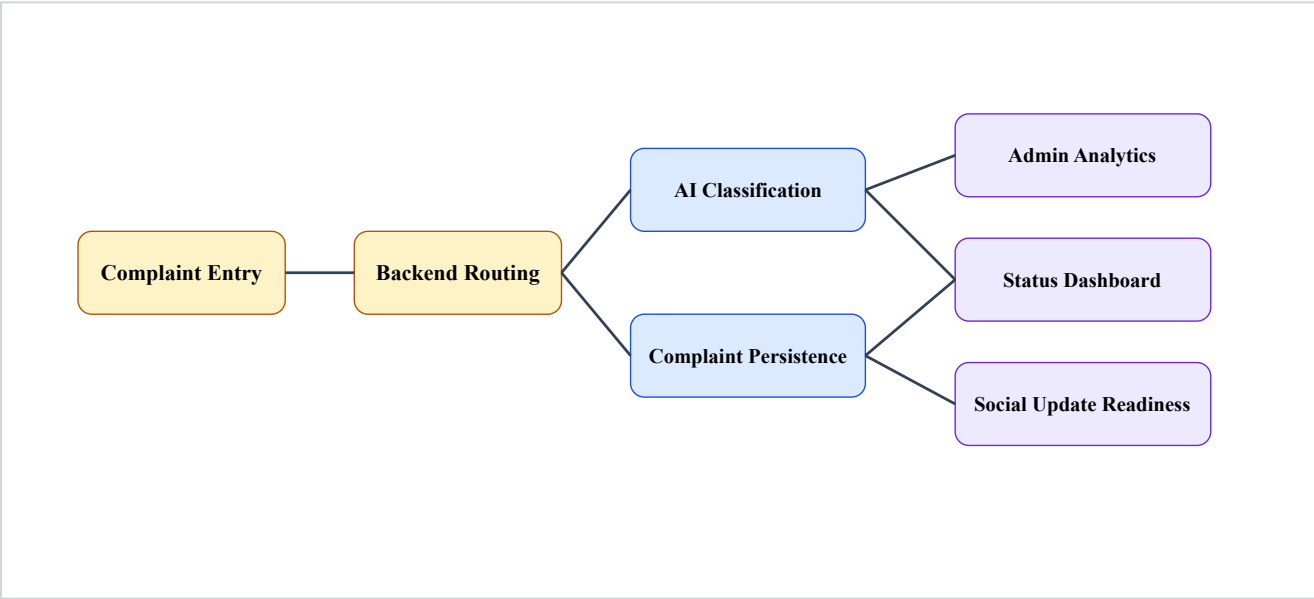


Figure 4.2.2: Module Interaction Diagram

4.3 Flow Chart / Activity Diagram

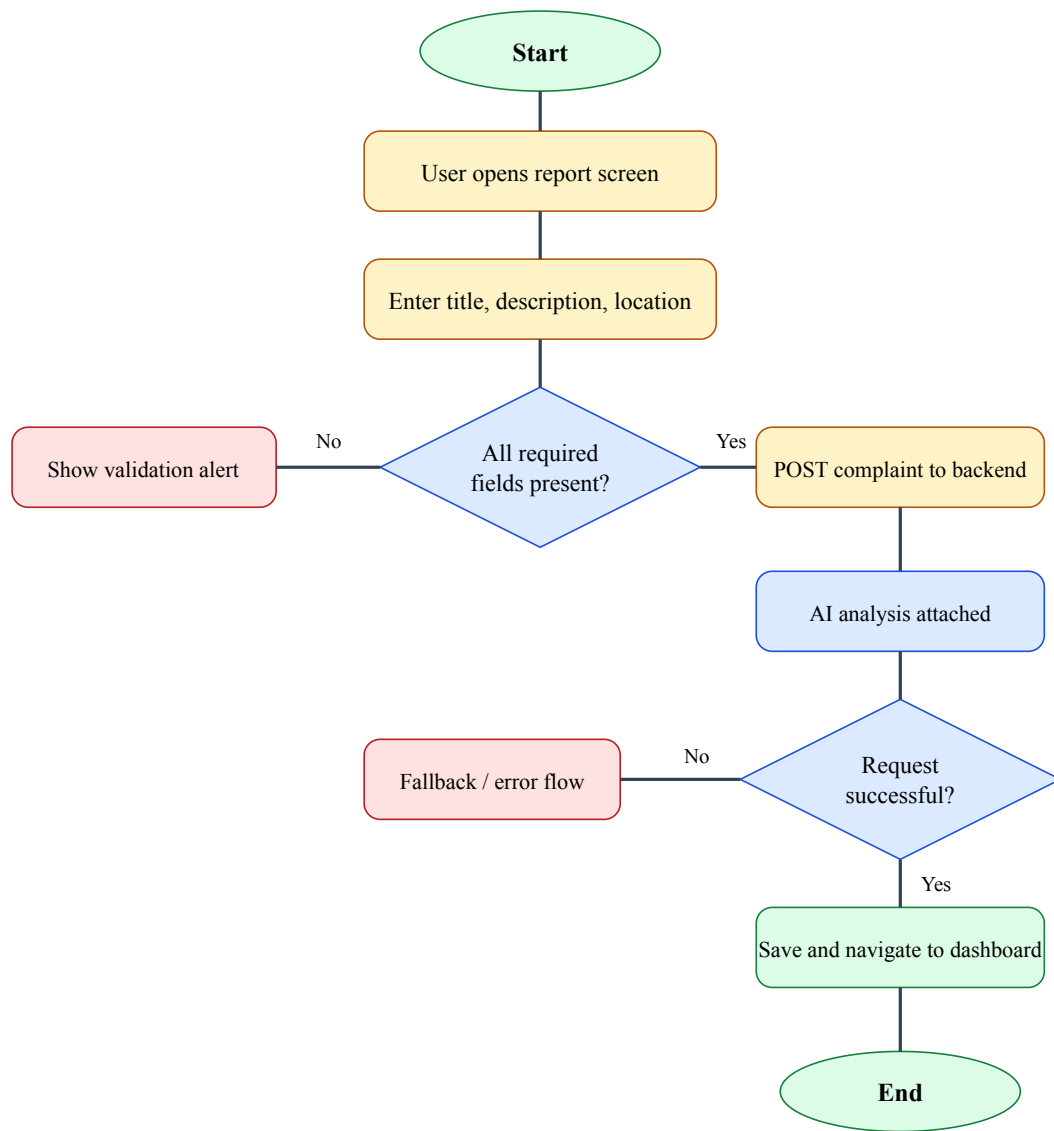


Figure 4.3.1: Activity Diagram for Complaint Submission

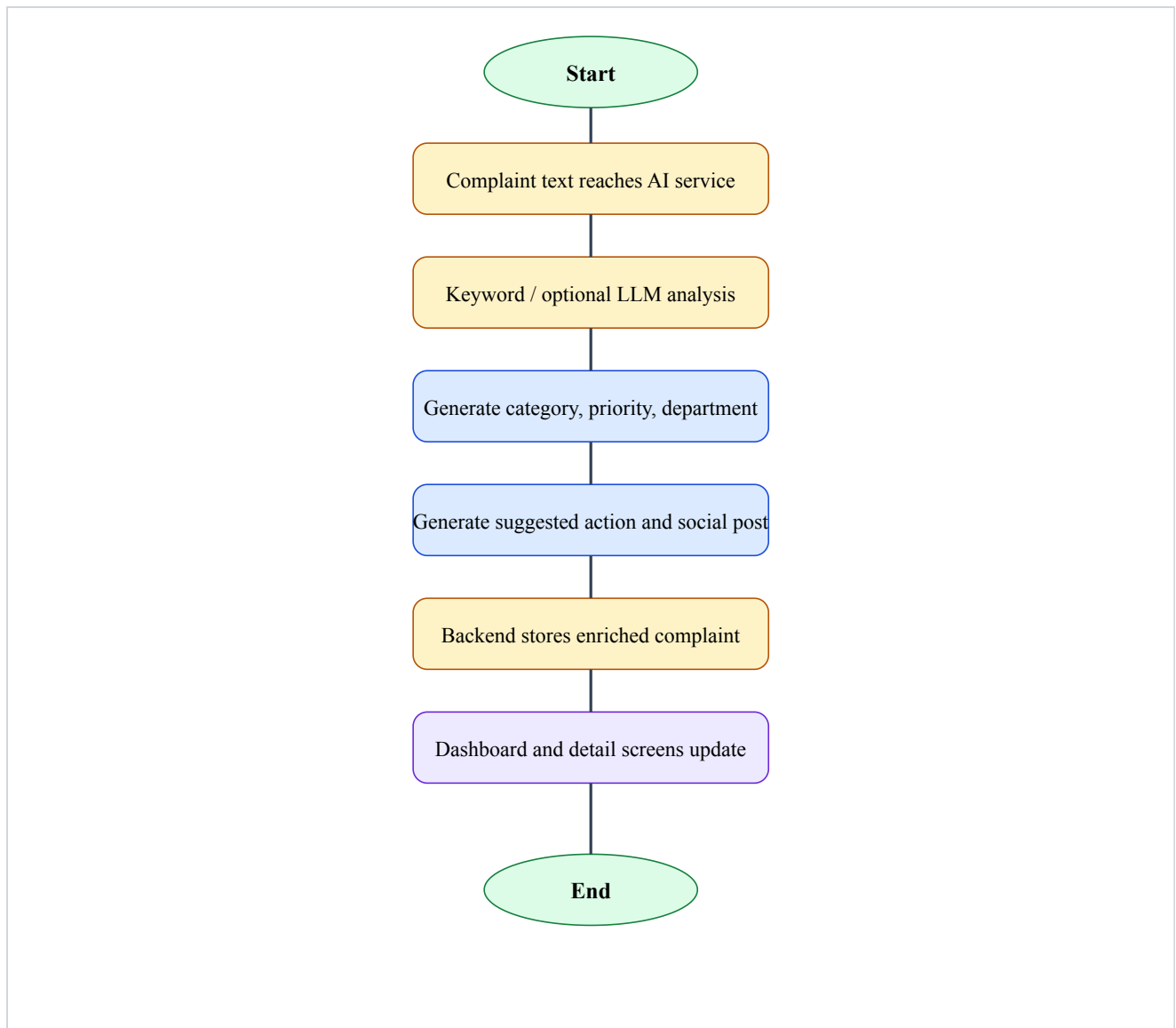


Figure 4.3.2: Activity Diagram for AI Analysis and Status Tracking

4.4 ER Diagram / Class Diagram

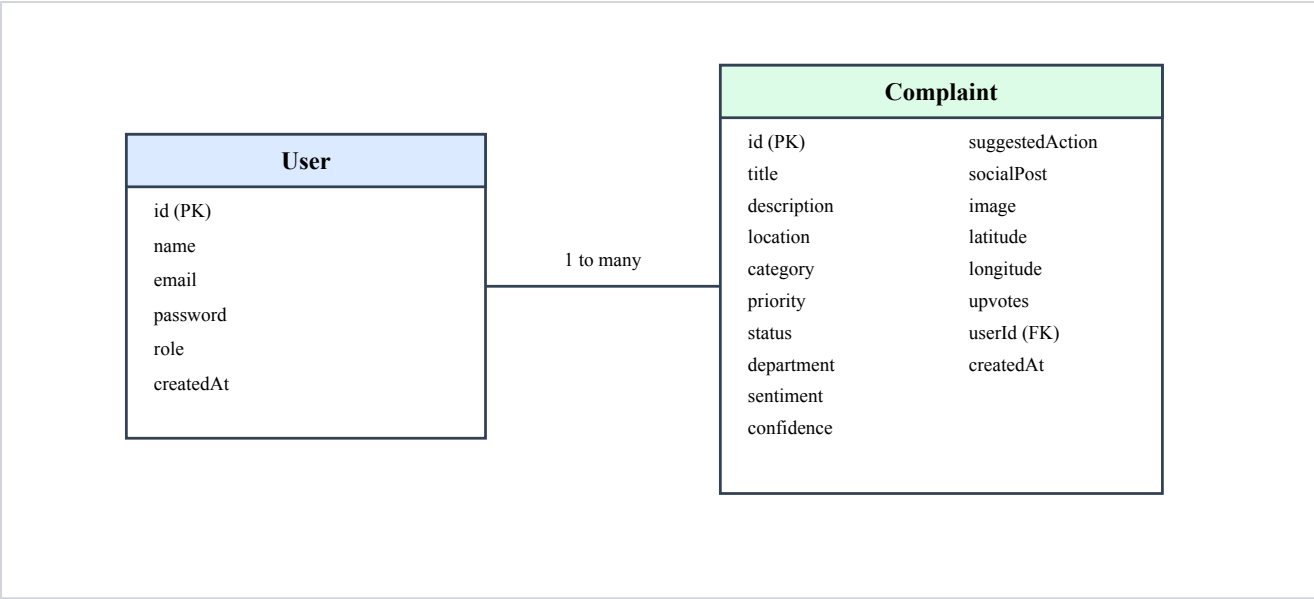


Figure 4.4.1: ER Diagram

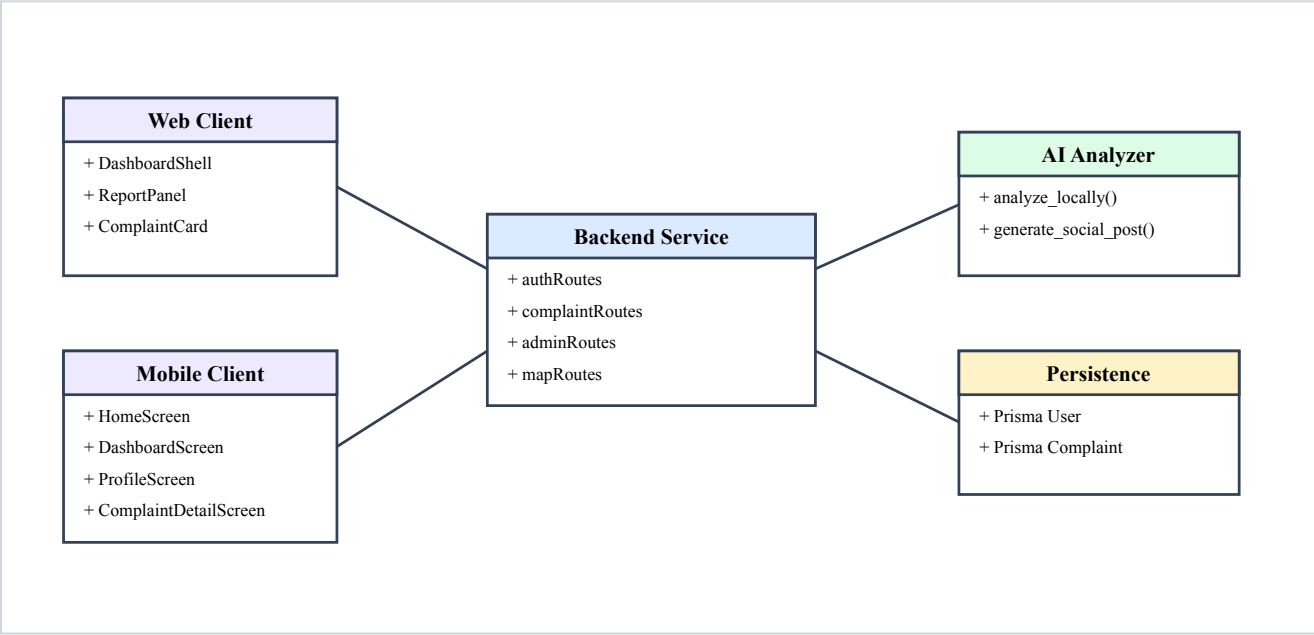


Figure 4.4.2: Class / Data Model Diagram

CHAPTER 5: IMPLEMENTATION

5.1 Application Screens and Working

Web Application

The web application is built in Next.js and functions as the primary dashboard-oriented interface. The landing page presents the civic issue resolution system with complaint snapshots and quick navigation to registration and the operational dashboard. The dashboard combines a reporting panel, complaint cards, hotspot summaries, and administrative overview metrics. This makes the web view suitable for a supervisory or administrative workflow in which multiple complaints must be scanned in a short time.

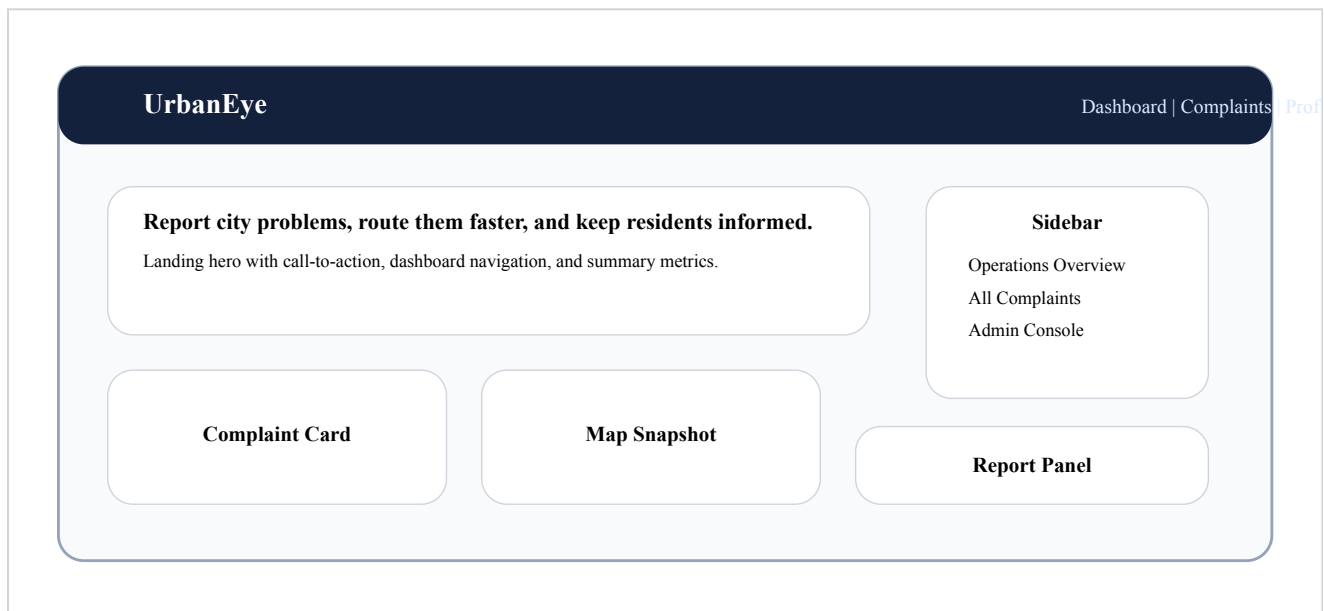


Figure 5.1.1: Web Home and Dashboard Flow

Mobile Application

The mobile application is designed from the citizen's perspective. Its screens focus on quick complaint entry, meaningful status visibility, and reduced friction in follow-up. The home screen now highlights complaint counts and X-account readiness, the dashboard emphasizes the proportion of solved and open complaints, and the profile screen stores a preferred X handle. Local complaint persistence is used so that the dashboard remains informative even under fallback conditions.

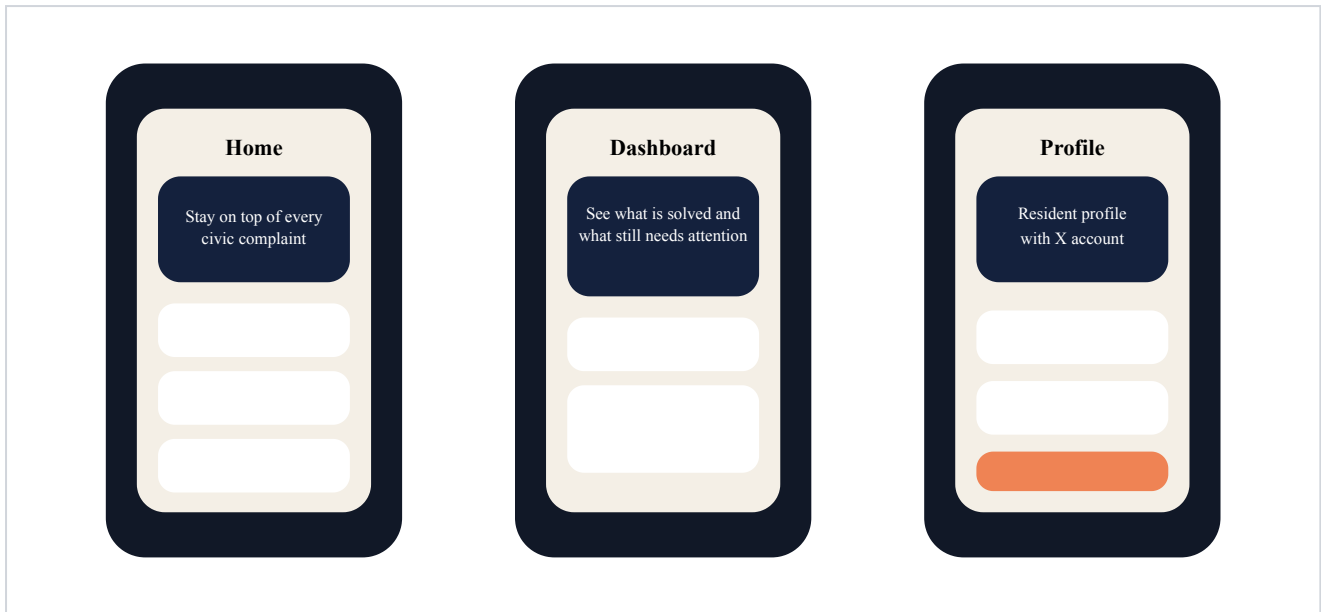


Figure 5.1.2: Mobile Home and Dashboard Flow

5.2 Source Code of Some Modules

Complaint Analysis Flow

The following backend-side AI logic illustrates how a complaint is enriched before being returned to the application. The system relies on keyword-based matching with optional LLM enhancement and then generates a compact social update that can be used for public communication.

```
def analyze_locally(payload: ComplaintPayload) -> AnalysisResult:
    text = f"{payload.title} {payload.description} {payload.location}".lower()

    category = "General"
    department = "Civic Response Cell"
    priority = "MEDIUM"
    sentiment = "concerned"

    if keyword_match(text, ["garbage", "waste", "trash"]):
        category = "Sanitation"
        department = "Sanitation Department"
        priority = "HIGH"
    elif keyword_match(text, ["light", "power", "electric"]):
        category = "Electricity"
        department = "Electricity Department"

    action = f"Route this complaint to {department} and request field validation."
    social = generate_social_post(...)
```

Backend Complaint Creation

```
router.post("/", createComplaint);
router.get("/", getComplaints);
router.get("/:id", getComplaint);

const analysis = await analyzeComplaint({ title, description, location });

const complaint = await prisma.complaint.create({
  data: {
    title: title.trim(),
    description: description.trim(),
    location: location.trim(),
    category: analysis.category,
    priority: analysis.priority,
    department: analysis.department,
    sentiment: analysis.sentiment,
    confidence: analysis.confidence,
    suggestedAction: analysis.suggestedAction,
    socialPost: analysis.socialPost,
  }
});
```

Mobile Complaint Tracking Logic

```
const stats = useMemo(() => getComplaintStats(data), [data]);
const filteredData = useMemo(() => filterComplaints(data, activeFilter),
[activeFilter, data]);
const solvedRate = stats.total ? Math.round((stats.resolved / stats.total) * 100)
: 0;

{DASHBOARD_FILTERS.map((filter) => {
  const isActive = activeFilter === filter.key;
  return (
    <TouchableOpacity key={filter.key} onPress={() =>
setActiveFilter(filter.key)}>
      ...
    </TouchableOpacity>
  );
})}
```


CHAPTER 6: TESTING

6.1 Testing Strategy

The testing strategy for UrbanEye was based on iterative verification of individual modules, API routes, and client interfaces. Since the project is composed of several independently runnable parts, each layer was checked using the most practical method available: syntax checks for source modules, bundling for frontend validation, complaint-flow API calls for backend behavior, and fallback execution for resilience testing.

6.2 Test Cases

Test Case	Input / Action	Expected Result	Observed Result
User Registration	Submit name, email, password	User record created or fallback user returned	Passed
Complaint Submission	Submit title, description, location	Complaint stored with status and analysis metadata	Passed
AI Analysis	Streetlight / garbage / water complaint text	Correct category and department inference	Passed
Social Post Generation	Complaint routed through AI service	X-ready text generated within compact length	Passed
Dashboard Filtering	Select Pending / In Progress / Solved	Complaint list filtered correctly	Passed
Mobile Local Persistence	Create complaint in fallback mode	Complaint remains visible after reload	Passed

6.3 Validation Methods Used

- Python syntax compilation for the FastAPI service modules.
- Node syntax checks for the backend and JavaScript service files.
- Prisma client generation to validate schema consistency.
- Expo Android bundling to ensure the mobile application compiles successfully.
- Next.js development server execution to validate web frontend startup.

6.4 Testing Summary

The application components were found to be functionally coherent for the intended project scope. The backend routes handled both persistent and fallback complaint flows correctly. The AI service generated structured response objects that matched the frontend expectations. The mobile dashboard successfully highlighted solved versus open complaints, while the web dashboard consolidated complaint reporting and oversight. The most notable strength of the test cycle was the presence of fallback mechanisms, which ensured the product remained demonstrable even in a constrained environment.

CHAPTER 7: SUMMARY AND CONCLUSION

7.1 Summary of Work Done

UrbanEye was developed as an integrated civic complaint resolution platform with four major technical layers: web frontend, mobile frontend, backend API, and AI service. The project began with the core requirement of complaint registration and gradually evolved into a more complete civic workflow system that supports complaint analysis, structured routing, resident status tracking, and administrative monitoring.

On the backend side, complaint, authentication, map, and admin routes were organized in Express. The persistence layer was prepared through Prisma models for user and complaint entities, while a mock store ensured that the system remained operable in demonstration mode. On the AI side, a FastAPI microservice was used to generate category, priority, department, sentiment, suggested action, and X-ready update text. The AI flow was deliberately grounded in practical routing utility rather than purely experimental text generation.

The web application presented complaint dashboards and operational summaries, whereas the mobile application focused on a user-friendly citizen journey with complaint reporting, improved status interpretation, and profile-level X account preferences. Together, these components formed a cohesive project that demonstrates the use of full-stack engineering and assistive AI in public-service workflows.

7.2 Conclusion

The dissertation objective of building an AI-assisted civic issue resolution system has been successfully met in the form of UrbanEye. The project demonstrates that even a relatively compact academic system can meaningfully improve complaint handling when structured data, dashboard visibility, and AI-assisted triage are combined. Rather than simply storing complaint text, the system transforms input into actionable and trackable information.

The strongest contribution of the work lies in its workflow integration. Complaint collection, analysis, storage, dashboarding, and communication preparation are not isolated features; they are linked parts of the same service pipeline. This makes UrbanEye a useful foundation for future civic-tech expansion and a strong example of applied software engineering in a public-facing problem domain.

CHAPTER 8: LIMITATIONS AND FUTURE WORK

8.1 Limitations of the Project

1. The current complaint intelligence primarily uses rule-based local analysis, with optional LLM enhancement rather than a fully trained domain model.
2. The complaint status lifecycle is simplified and does not yet include a complete department-side action workflow or staff assignment engine.
3. The map hotspot layer is intentionally lightweight and not yet connected to a full GIS visualization stack.
4. The current system supports X-account preference and X-ready content generation, but not direct authenticated publishing to the X platform.
5. Fallback demo data is still used in some flows to keep the project resilient in local development, which means not every interface always represents real production traffic.
6. The report cover page still requires final institutional details such as supervisor name, department, and enrollment number before university submission.

8.2 Future Scope

1. Integration with official civic department dashboards and authenticated authority accounts.
2. Real-time notifications through email, SMS, and push notification services.
3. Direct X publishing for approved official communication after human confirmation.
4. Geospatial clustering and heatmap-based issue prioritization for city-scale deployments.
5. Machine-learning models trained on historical civic complaints to improve categorization and urgency scoring.
6. Complaint assignment and field staff tracking for resolution accountability.
7. Multilingual complaint intake for wider accessibility.
8. Evidence-based analytics linking recurring hotspots to planning and maintenance policies.

BIBLIOGRAPHY

1. Next.js Documentation. Vercel. Available at: <https://nextjs.org/docs>
2. React Documentation. Meta. Available at: <https://react.dev>
3. Express.js Documentation. Available at: <https://expressjs.com>
4. FastAPI Documentation. Available at: <https://fastapi.tiangolo.com>
5. Prisma ORM Documentation. Available at: <https://www.prisma.io/docs>
6. Expo Documentation. Available at: <https://docs.expo.dev>
7. LangChain Documentation. Available at: <https://python.langchain.com>
8. PostgreSQL Documentation. Available at: <https://www.postgresql.org/docs>
9. Bcrypt Documentation and community references for password hashing.
10. General software engineering references used for requirement analysis, system design, and workflow modeling.

APPENDIX

A. Important API Endpoints

Method	Endpoint	Purpose
POST	/api/auth/register	Register a new user
POST	/api/auth/login	Login an existing user
POST	/api/complaints	Create a complaint and attach AI analysis
GET	/api/complaints	List complaints
GET	/api/complaints/:id	Get complaint details
GET	/api/admin/overview	Administrative statistics summary
GET	/api/admin/complaints	Administrative complaint list
GET	/api/admin/users	Administrative user list
GET	/api/map/hotspots	Complaint hotspot information
POST	/analyze	AI microservice complaint analysis

B. Note on Final Submission Editing

Before formal university submission, the student should replace placeholder items on the cover and certificate pages with the final supervisor name, institution details, enrollment number, and any official formatting elements required by the department. The technical content, diagrams, and chapter structure in this report have been aligned to the dissertation format recovered from the provided PDF template.